



UNIVERSIDADE DO ESTADO DE
SANTA CATARINA - UDESC
CENTRO DE CIÊNCIAS TECNOLÓGICAS - CCT

Estrutura de dados I

Relatório Fila

Alunos(as): Amanda Biancatti e Guilherme Hoerning Reinert
Professor: Gilmaro Barbosa Dos Santos

Joinville, 2025.

Sumário

1. Introdução.....	pag. 03
2. Objetivos.....	pag. 03
3. Metodologia.....	pag. 04
3.1 Recursos para o experimento.....	pag. 04
3.2 Procedimento Experimental.....	pag. 05
4. Resultados e discussões.....	pag. 06
5. Conclusão.....	pag. 10
6. Referências bibliográficas.....	pag. 12

1) INTRODUÇÃO

Uma **fila de prioridade** organiza elementos conforme sua importância, e não pela ordem de chegada. Existem duas versões analisadas:

- **Sem referencial móvel:** percorre sempre a fila desde a cauda até encontrar a posição correta.
- **Com referencial móvel:** utiliza um ponteiro auxiliar (ref) para otimizar o caminho de inserção, começando do ponto mais próximo.

O **custo computacional** refere-se ao esforço necessário para executar operações, como inserções. Para analisá-lo, contamos quantas vezes a **função de comparação** é chamada, pois ela representa o trabalho para posicionar um elemento corretamente.

O uso de **conjuntos de dados definidos** é essencial para garantir uma comparação justa entre as filas. Esses dados permitem medir o desempenho das inserções em diferentes situações.

A **quantificação do custo** é feita com base no número de comparações, que indica o esforço computacional envolvido. Esse valor é um bom indicador porque reflete diretamente a eficiência do algoritmo de inserção em cada tipo de fila.

2) Objetivos

Tarefa 1B: FDE - Prioridade

Objetivo: Análise comparativa de desempenho entre duas implementações de Fila Duplamente Encadeada de Prioridade (com e sem referencial móvel), usando dados extraídos do *dataset*. A comparação visa destacar os benefícios de otimizações na estrutura de dados, especialmente ao tempo de inserção e remoção de elementos.

Em geral:

- Implementar a FDE de prioridade com referencial móvel.
- Comparar o número médio de iterações de inserção nas duas implementações;
- Avaliar o impacto do uso do referencial móvel no desempenho da estrutura.
- Medir o desempenho com diferentes quantidades de dados;

- Observar os efeitos da variação do atributo de prioridade (*ranking* e matrícula);
- Gerar gráficos comparativos entre as implementações;
- Interpretar os resultados e justificar os comportamentos observados;
- Verificar quais bases favorecem ou prejudicam o desempenho da FDE com referencial móvel.

3) Metodologia

3.1) Recursos para o experimento

Foi realizado o desenvolvimento e análise de desempenho de duas versões de uma fila duplamente encadeada de prioridade:

- FDE sem referencial móvel;
- FDE com referencial móvel;

Ambas utilizam o campo *ranking* como critério de prioridade.

A entrada de dados consiste em 10.000 registros do arquivo 'dataset_v1.csv'. Os testes consistem em inserir subconjuntos com tamanhos crescentes (de 500 a 9000 em passos de 500), medindo o número médio de iterações necessárias para cada inserção em ambas as implementações.

Estrutura “info” - Cada elemento da fila representa uma pessoa e seus dados associados. Essa estrutura contém nome (uma string que armazena o nome da pessoa), matrícula (um inteiro que funciona como identificador único), *ranking* (um inteiro usado para determinar a prioridade dentro da fila) e o curso (uma string indicando o curso da pessoa).

Esses campos são fundamentais para a ordenação e manipulação dos nodos na fila.

Estrutura Nodo – Para montar a fila, cada elemento é um nodo, que contém os dados da pessoa (guardados na estrutura “info”), um ponteiro para o nodo posterior (permitindo percorrer a fila de trás para frente) e um ponteiro para o nodo anterior (permitindo a fila de frente para trás).

Isso cria uma estrutura de fila duplamente encadeada, facilitando inserções e remoções em qualquer posição da fila.

3.2) Procedimento Experimental

O programa principal está estruturado em torno de um **menu interativo**, acessado via terminal, que permite ao usuário realizar uma série de operações com base nos dados carregados do CSV. Todas as operações manipulam uma FDE-prioridade com referencial móvel, sendo que a fila sem referencial móvel é utilizada somente na etapa de comparação de desempenho.

As principais etapas são:

Ao iniciar, o programa lê todas as linhas do arquivo CSV e as armazena em memória como um vetor de strings. O usuário pode escolher quantos registros do dataset deseja usar como base. O programa então seleciona esses dados aleatoriamente para manipulação. É possível exibir na tela os dados escolhidos.

Um descritor é criado, representando a estrutura da fila com ponteiros para o início, fim, referencial móvel, tamanho e outros dados auxiliares. O usuário escolhe um índice da base de dados e insere esse item na fila. A inserção é feita com base em um critério de prioridade (ex: *ranking*). O programa permite remover o elemento com maior prioridade da fila. É possível buscar e exibir o primeiro ou o último nodo da fila, além de visualizar todos os elementos. Remove todos os nodos da fila, zerando-a.

Na fila com referencial móvel, é uma versão otimizada, o algoritmo mantém um ponteiro de referência que se move ao longo da estrutura conforme os dados são inseridos. Isso reduz o número de comparações necessárias para encontrar a posição correta de inserção com base na prioridade.

Na fila sem referencial móvel, uma versão mais básica, toda inserção ou remoção percorre a fila sequencialmente sempre pela cauda ou pela frente, comparando item por item até encontrar o ponto certo.

O código também permite comparar o número de repetições de laço entre as filas de prioridade em *ranking* ou matrícula, com o intuito de utilizar este dado para definir o desempenho de ambas as filas. A comparação entre as FDE-prioridade com e sem referencial móvel é ilustrada ao usuário por meio de duas tabelas: uma usando o *ranking* como critério de comparação; e a outra utilizando a matrícula de cada registro.

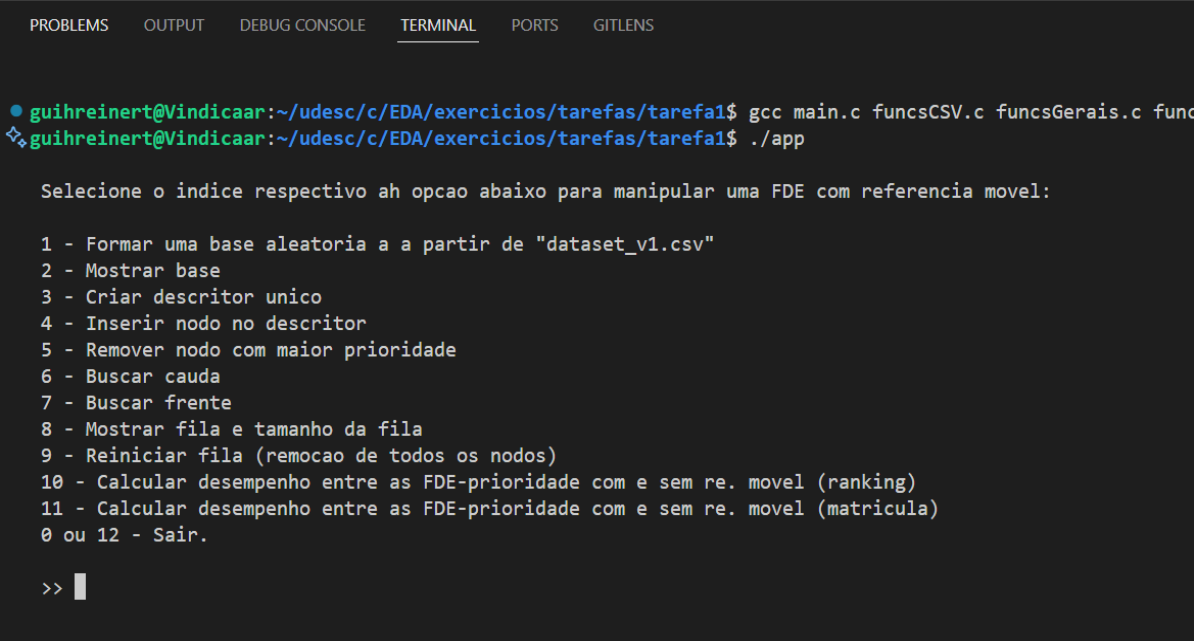
Cada tabela apresenta os seguintes dados: Base utilizada; tipo de fila; número de repetições realizadas; a média destas repetições em cada base; a razão entre o número de repetições entre FDE-prioridade sem referencial móvel e com referencial móvel, respectivamente; e o inverso, a razão entre a fila com referencial móvel e sem referencial móvel. Ambas estas razões não foram pedidas para constarem nas análises do código, mas ainda assim são

úteis para expor a otimização oferecida pela inserção com base no referencial móvel, estabelecendo uma relação direta entre os dados de cada fila.

4) Resultados e análise

O código da inserção de elementos em uma FDE-prioridade com referencial móvel aqui produzido é capaz de diminuir em média 3 vezes o número de repetições de laço na inserção de nodo em uma fila sem referencial móvel.

A fila de prioridade com referencial móvel se destaca pela eficiência em inserções, principalmente em filas longas. Apesar da complexidade adicional, o uso do campo “ref” como ponto intermediário de busca reduz significativamente o número de comparações necessárias, otimizando o desempenho da estrutura. A versão sem referencial, por outro lado, é mais direta e mais simples de implementar, sendo ideal para filas pequenas ou quando a simplicidade do código for mais relevante que a performance.



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  GITLENS

guihreinert@Vindicaar:~/udesc/c/EDA/exercicios/tarefas/tarefa1$ gcc main.c funcsCSV.c funcsGerais.c func
guihreinert@Vindicaar:~/udesc/c/EDA/exercicios/tarefas/tarefa1$ ./app

Selecione o indice respectivo ah opcao abaixo para manipular uma FDE com referencia movel:

1 - Formar uma base aleatoria a a partir de "dataset_v1.csv"
2 - Mostrar base
3 - Criar descritor unico
4 - Inserir nodo no descritor
5 - Remover nodo com maior prioridade
6 - Buscar cauda
7 - Buscar frente
8 - Mostrar fila e tamanho da fila
9 - Reiniciar fila (remocao de todos os nodos)
10 - Calcular desempenho entre as FDE-prioridade com e sem re. movel (ranking)
11 - Calcular desempenho entre as FDE-prioridade com e sem re. movel (matricula)
0 ou 12 - Sair.

>> █
```

Figura 1. Printscreen do menu de opções gerado pelo terminal.

PROBLEMS	OUTPUT	DEBUG CONSOLE	TERMINAL	PORTS	GITLENS
INSERCAO POR RANKING					
Base	Fila	Num Repeticoes	Media Repeticoes	numRep_Sem / numRep_Com	numRep_Com / numRep_Sem
500	SEM REF: 60021 COM REF: 19988	120.04 39.98	3.00	33.30%	
1000	SEM REF: 242183 COM REF: 81231	242.18 81.23	2.98	33.54%	
1500	SEM REF: 549284 COM REF: 187515	366.19 125.01	2.93	34.14%	
2000	SEM REF: 977619 COM REF: 334929	488.81 167.46	2.92	34.26%	
2500	SEM REF: 1515074 COM REF: 519056	606.03 207.62	2.92	34.26%	
3000	SEM REF: 2183109 COM REF: 737491	727.70 245.83	2.96	33.78%	
3500	SEM REF: 2987040 COM REF: 996249	853.44 284.64	3.00	33.35%	
4000	SEM REF: 3948158 COM REF: 1305339	987.04 326.33	3.02	33.06%	
4500	SEM REF: 4976915 COM REF: 1648788	1105.98 366.40	3.02	33.13%	
5000	SEM REF: 6122649 COM REF: 2056021	1224.53 411.20	2.98	33.58%	
5500	SEM REF: 7401921 COM REF: 2486978	1345.80 452.18	2.98	33.60%	
6000	SEM REF: 8779898 COM REF: 2958879	1463.32 493.15	2.97	33.70%	
6500	SEM REF: 10341834 COM REF: 3475111	1591.05 534.63	2.98	33.60%	
7000	SEM REF: 12024870 COM REF: 4031776	1717.84 575.97	2.98	33.53%	
7500	SEM REF: 13698531 COM REF: 4623372	1826.47 616.45	2.96	33.75%	
8000	SEM REF: 15545886 COM REF: 5273134	1943.24 659.14	2.95	33.92%	
8500	SEM REF: 17972748 COM REF: 5938193	2114.44 698.61	3.03	33.04%	
9000	SEM REF: 20173218 COM REF: 6658002	2241.47 739.78	3.03	33.00%	
Tempo de execucao: 1.797159s					
>>					

Figura 2. Printscreen da tabela dos resultados do total de número de repetições realizadas para a inserção de nodos em cada base pelo critério de ranking e dados associados.

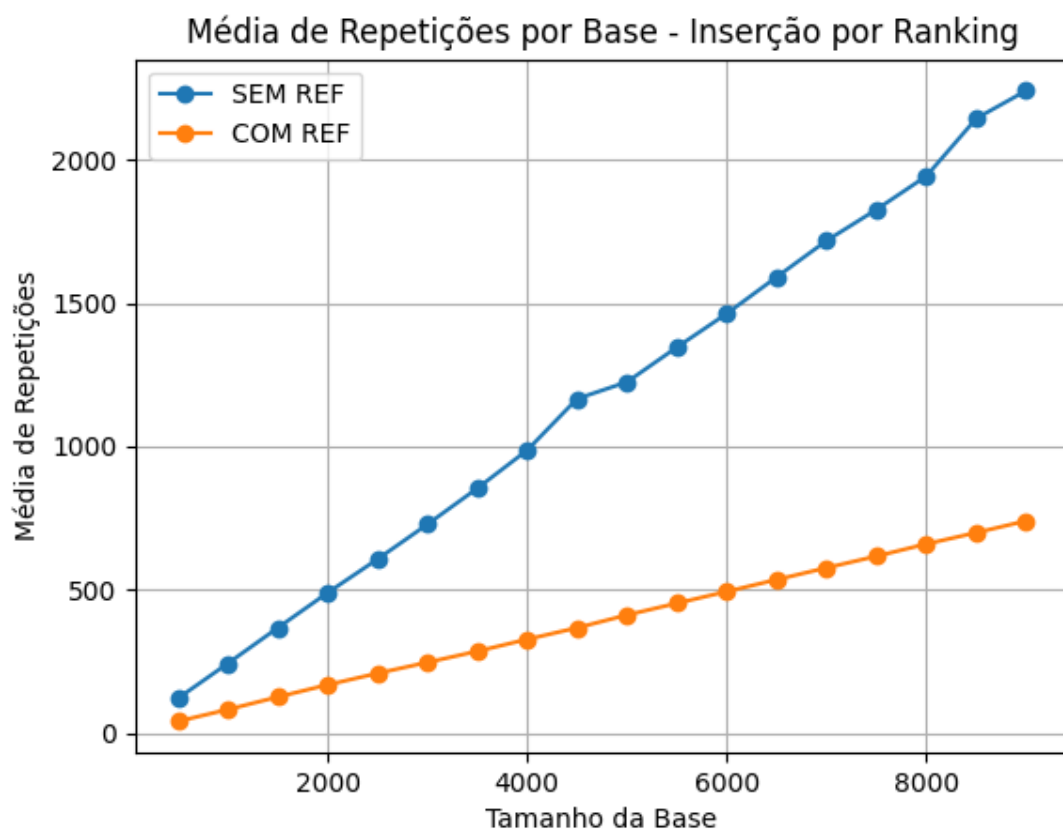


Figura 3. Média de repetições nas inserções em cada fila por base, tendo o ranking como critério de prioridade.

PROBLEMS	OUTPUT	DEBUG CONSOLE	TERMINAL	PORTS	GITLENS
INSERCAO POR MATRICULA					
Base	Fila	Num Repeticoes	Media Repeticoes	numRep_Sem / numRep_Com	numRep_Com / numRep_Sem
500	SEM REF:	61669	123.34	2.98	
	COM REF:	20724	41.45		33.61%
1000	SEM REF:	260442	260.44	3.12	
	COM REF:	83559	83.56		32.08%
1500	SEM REF:	570660	380.44	3.06	
	COM REF:	186717	124.48		32.72%
2000	SEM REF:	1003385	501.69	3.00	
	COM REF:	334230	167.12		33.31%
2500	SEM REF:	1584984	633.99	2.99	
	COM REF:	529212	211.68		33.39%
3000	SEM REF:	2278601	759.53	3.01	
	COM REF:	757522	252.51		33.25%
3500	SEM REF:	3123810	892.52	3.00	
	COM REF:	1040063	297.16		33.29%
4000	SEM REF:	4093084	1023.27	3.02	
	COM REF:	1354635	338.66		33.10%
4500	SEM REF:	5122115	1138.25	2.97	
	COM REF:	1725804	383.51		33.69%
5000	SEM REF:	6299152	1259.83	2.98	
	COM REF:	2112053	422.41		33.53%
5500	SEM REF:	7652202	1391.31	2.99	
	COM REF:	2555726	464.68		33.40%
6000	SEM REF:	9055656	1509.28	2.97	
	COM REF:	3045898	507.65		33.64%
6500	SEM REF:	10638799	1636.74	2.97	
	COM REF:	3582244	551.11		33.67%
7000	SEM REF:	12389369	1769.91	2.99	
	COM REF:	4141102	591.59		33.42%
7500	SEM REF:	14214453	1895.26	3.00	
	COM REF:	4730789	630.77		33.28%
8000	SEM REF:	16107847	2013.48	3.00	
	COM REF:	5376366	672.05		33.38%
8500	SEM REF:	18106068	2130.13	2.97	
	COM REF:	6095422	717.11		33.67%
9000	SEM REF:	20301383	2255.71	2.98	
	COM REF:	6812367	756.93		33.56%
Tempo de execucao: 1.721070s					
>>					

Figura 4. Printscreen da tabela dos resultados do total de número de repetições realizadas para a inserção de nodos em cada base pelo critério de matrícula e dados associados.

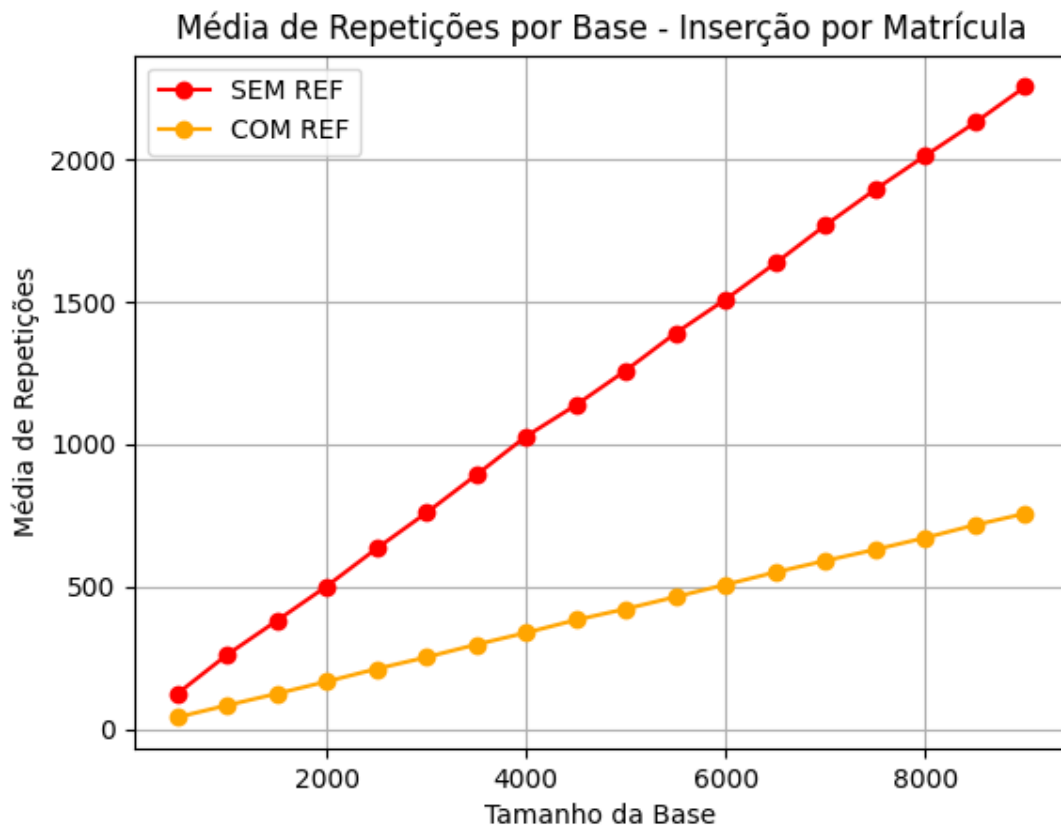


Figura 5. Média de repetições nas inserções em cada fila por base, tendo a matrícula como critério de prioridade.

5) Conclusão

Em geral, analisamos comparativamente o desempenho entre duas implementações da FDE de prioridade (com e sem referencial móvel), utilizando dados extraídos do dataset.

Para cada tópico do item 2:

1) A implementação mostrou-se tecnicamente viável e eficiente. A adição de um ponteiro de referência dinâmico dentro do descritor permitiu reduzir o custo de percorrimento da lista ao realizar inserções com base na prioridade, sem alterar a lógica de remoção. A estrutura se manteve organizada e compatível com as demais operações da FDE tradicional.

2) Os dados coletados mostraram uma diminuição consistente no número médio de iterações realizadas pela FDE com referencial móvel. Essa redução foi mais perceptível em listas maiores, onde a busca pela posição ideal de inserção se tornaria custosa na versão sem referencial. Em listas menores, a diferença foi menor, mas ainda presente.

3) O uso do referencial móvel teve impacto positivo direto no desempenho geral da estrutura, principalmente nas operações de inserção. Como o referencial é atualizado com base na posição anterior de inserção, a fila consegue evitar buscas desnecessárias em casos em que os novos dados seguem um padrão de prioridade relativamente previsível. Isso resultou em ganho de desempenho sem comprometer a integridade da fila.

4) Os testes realizados com volumes crescentes de dados revelaram que a FDE com referencial móvel escala melhor do que a versão tradicional. À medida que o número de elementos aumenta, a diferença de desempenho se amplia em favor da versão otimizada. Isso confirma que o referencial móvel é uma melhoria especialmente eficaz para aplicações que lidam com grandes quantidades de dados.

5) Ao utilizar diferentes critérios de prioridade, observou-se que o desempenho varia de acordo com a distribuição dos dados. Quando o atributo de prioridade apresentava maior variação (como o *ranking*), o ganho de desempenho era mais evidente. Já com atributos mais uniformes (como matrícula sequencial), a vantagem do referencial se mostrava menos expressiva.

6) Os gráficos gerados a partir dos testes empíricos reforçaram visualmente os resultados observados nas métricas. Comparações lado a lado mostraram reduções claras no tempo de inserção e no número de iterações, destacando a eficiência da versão com referencial móvel. Esses dados foram essenciais para justificar o ganho de desempenho da estrutura otimizada.

7) A análise dos resultados sugere que o comportamento mais eficiente da FDE com referencial móvel está relacionado à sua capacidade de “lembrar” a última posição acessada e usá-la como ponto de partida inteligente. Essa característica é especialmente útil em cenários onde os dados chegam com prioridades que têm relação entre si. Em contrapartida, quando as prioridades são altamente aleatórias, o referencial pode perder eficiência relativa.

8) Por fim, verificou-se que bases de dados com ampla dispersão de prioridade ou inserções em ordem crescente ou decrescente favorecem fortemente a utilização do referencial móvel. Já em bases com valores repetitivos ou pouca dispersão, a otimização tem impacto limitado. Essa observação é crucial para orientar o uso prático da estrutura, sugerindo sua aplicação em sistemas com entrada de dados não totalmente aleatória.

6) Referências bibliográficas

- TENEMBAUM, Aaron M. et al. Estruturas de Dados Usando C. Ed. Makron Books.
- HOROWITZ, Ellis. & SAHNI, Sartaj. Fundamentos de Estruturas de Dados. Editora Campus.
- SZWARCFITER, J. L. et al. Estruturas de Dados e seus Algoritmos. Ed. LTC.